

Edge cases (200 doesn't mean safe to trust) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real response-parser handles shapes that are wrong in less obvious ways). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

There are two separate problems to demonstrate here, and they need two separate pieces of code.

Problem 1: the response body isn't valid JSON at all — maybe it's an HTML error page. Calling `.json()` on that raises an error, and you should catch it specifically, not let it crash your program.

Problem 2: the body *is* valid JSON, but a key your code expected just isn't there. Python gives you two different ways to read a dictionary key — one that crashes if it's missing, and one that lets you supply a fallback instead.

Things to use:

- `import json` and `json.loads("this is not json")` — raises `json.JSONDecodeError`.
- `try: ... except json.JSONDecodeError as e: print(e)` — catch it and print what went wrong.
- `data = {"a": 1}` then `data.get("missing_key")` — prints `None`, no crash.
- `data["missing_key"]` — raises `KeyError` if you try this instead.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Problem 1 — invalid JSON: `response.json()` internally calls `json.loads()` on the body text and raises `json.JSONDecodeError` (technically `requests.exceptions.JSONDecodeError` when called through `requests`, which subclasses it) if the body can't be parsed at all. You don't need a real broken server to test this — build a fake object, or use `json.loads("not json")` directly.

Problem 2 — valid JSON, wrong shape: once you have a real `dict`, `some_dict["missing_key"]` raises `KeyError` immediately. `some_dict.get("missing_key")` instead returns `None` (or a default you supply: `.get("missing_key", "fallback")`) — no error at all. Neither one is universally "correct" — it depends whether a missing key is a bug you want to know about loudly, or a normal case you want to handle gracefully.

The exact pieces:

- `json.JSONDecodeError` — a subclass of `ValueError`, raised with a `.msg`, `.pos`, and a useful `str(e)` describing exactly where the parsing broke.
- `dict.get(key, default=None)` — never raises; returns `default` if the key isn't present.
- `dict[key]` — raises `KeyError` immediately if the key is missing; this is the "loud" option.
- **Choosing between them isn't random** — use `[key]` (or check with `in` first) for fields your code genuinely can't function without; use `.get(key, default)` for genuinely optional fields where a sensible fallback exists.

Sketch both blocks separately before checking Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

"Missing key" is only the simplest way valid JSON can still be the wrong shape. Think about two messier cases a real API can hand you: a key that's *present* but holds a completely different type than you expected (`response["choices"]` is a string instead of a list, because of an error path on the server's side), and a key that's buried several levels deep inside nested dictionaries and lists, where any one of those intermediate steps could be the thing that's missing (`response["choices"][0]["message"]["content"]` — 4 separate places this can fail).

`.get()` alone doesn't fully solve either problem: `data.get("choices")` returns `None` safely if "choices" is missing, but if it returns a string instead of a list, the very next line (`data["choices"][0]`) still crashes — just one line further downstream than before, at a spot with much less context about what actually went wrong. And chaining `.get()` calls for a nested path (`data.get("a", {}).get("b", {}).get("c")`) works, but silently returns `None` if any level is wrong, with no way to tell which level failed.

The real design question isn't "how do I read one key safely" — it's "how do I check an entire response's shape in one place, close to where it arrives, so a bad shape fails with one clear message naming exactly what's wrong, instead of a confusing `TypeError` or `IndexError` three functions later, far from the actual API call."

The extra pieces:

- `isinstance(value, list)` (or `dict`, `str`, etc.) — checking the *type* of a value you got back, not just whether the key existed.
- A small helper that walks a nested path and raises one clear, named error the moment anything along that path is missing or the wrong type — instead of letting Python's own `TypeError` / `IndexError` / `KeyError` surface wherever the code happens to touch the bad data next.

Sketch this nested-path helper yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both handle the single-level case — one key, either present with the right type or genuinely missing. Advanced adds the two ways that case doesn't cover: a key that's present but the *wrong type*, and a value buried several dictionary/list levels deep where any one level could be the problem — and argues for checking a response's whole expected shape in one place, right after parsing, instead of letting a bad shape surface as a confusing crash somewhere else in the program.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
part 1 - bad JSON:
    try:
        parse the text "this is not json" as JSON
    except JSON error as e:
        print the error

part 2 - missing key:
    make a dict with one key
    try .get("missing_key") -> see None, no crash
    try ["missing_key"] -> see it crash with KeyError
```

Here is almost the whole thing:

```
import json

try:
    json.loads("<html>not json</html>")
except json.JSONDecodeError as e:
    print("bad json:", e)
```

Expected output: bad json: Expecting value: line 1 column 1 (char 0) . What's missing: the missing-key demonstration with both `.get()` and `[...]` . Add it yourself, then check the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
import json

try:
    json.loads("this is not json")
except json.JSONDecodeError as e:
    print(f"invalid JSON: {e}")

data = {"choices": []}
print(data.get("message"))          # None, no crash

try:
    data["message"]
except KeyError as e:
    print(f"missing key: {e}")
```

```
import json

try:
    json.loads("<html>not json</html>")
except json.JSONDecodeError as e:
    print(f"bad json: {e}")

data: dict = {"choices": []}
print(data.get("message")) # safe, prints None
```

What's missing: the crashing version of the missing-key lookup, wrapped so it doesn't kill the script. Write it yourself, then compare against the [Solution](#). Notice both branches of problem 2 run — first the safe `.get()`, then, separately, the crashing `[...]` wrapped in its own `try/except` so it doesn't stop the script.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function get_nested(data, path, expected_type):
    current = data
    walked = ""
    for key in path:
        walked += "." + str(key)
        if key is a string and current is a dict and key in current:
            current = current[key]
        elif key is an int and current is a list and key < len(current):
            current = current[key]
        else:
            raise ValueError("missing at " + walked)

    if not isinstance(current, expected_type):
        raise ValueError("wrong type at " + walked + ", expected " + str(expected_type))

    return current
```

Here's almost the whole thing — fill in the missing piece yourself:

```
from typing import Any

def get_nested(data: Any, path: list, expected_type: type) -> Any:
    current = data
    walked_so_far = ""

    for key in path:
        walked_so_far += f".{key}"
        if isinstance(key, str) and isinstance(current, dict) and key in current:
            current = current[key]
        elif isinstance(key, int) and isinstance(current, list) and key < len(current):
            current = current[key]
        else:
            raise ValueError(f"expected response shape missing at {walked_so_far}")

    # your turn: check isinstance(current, expected_type) here, and raise
    # a clear ValueError naming walked_so_far and expected_type if it's wrong
    ...
```

```
return current
```

```
payload = {"choices": [{"message": {"content": "hi"}}]}  
content = get_nested(payload, ["choices", 0, "message", "content"], str)  
print(content)
```

Fill in the type check yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both demonstrate one key, one level deep, either present or genuinely missing. Advanced generalizes that into a single helper that walks an arbitrary path through nested dicts and lists, checking both "does each step along the way exist" and "is the final value the type I actually expected" — failing with one clear message naming exactly which step broke, instead of a `TypeError` or `IndexError` popping up wherever the bad data happens to get touched next.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)